NS3 NETWORK SIMULATION PROJECT

TCP Congestion Control for Bandwidth Reservation Networks

BR-TCP: Implementation, Simulation & Performance Evaluation using NS-3

Course	Network Simulation (NS3) — CSE 322
Student	Md. Dulal Hossain
Roll	2105169
Department	Computer Science & Engineering
Base Paper	Komuro et al., IEEE ICC 2006
Simulator	NS-3 (Network Simulator 3)
Networks	IEEE 802.11g (Static) & IEEE 802.15.4 (Static)
Date	April 2026

This report describes the full implementation and performance evaluation of BR-TCP — a bandwidth-reservation-aware TCP congestion control algorithm — simulated across wired and IEEE 802.11g / 802.15.4 wireless network environments using NS-3.

. Table of Contents

1. **Introduction & Motivation**
2. **Base Paper Summary**
3. **BR-TCP Algorithm Description**
4. **Simulator: NS-3 Overview**
5. **Network Topologies Under Simulation**
   - **5.1** Wired Network
   - **5.2** Wireless IEEE 802.11g Static
   - **5.3** Wireless IEEE 802.15.4 Static
6. **Parameters Under Variation**
7. **Modifications Made in the Simulator**
8. **Simulation Results & Graphs**
   - **8.1** Wired — BR-TCP Baseline Reproduction (Figs 4–8)
   - **8.2** Wired — Testbed-Equivalent Comparison (Figs 10–14)
   - **8.3** Improvement Graphs — Enhanced BR-TCP
   - **8.4** Wireless IEEE 802.11g Static
   - **8.5** Wireless IEEE 802.15.4 Static
9. **Summary Findings**
10. **Reflection on Findings**
11. **Conclusion**
12. **References**

1. Introduction & Motivation

High-speed networks built on **Multi-Protocol Label Switching (MPLS)** with bandwidth reservation offer a guaranteed minimum usable bandwidth per connection. However, conventional TCP congestion control algorithms — TCP Tahoe and TCP Reno — were designed for best-effort networks and react to packet loss by drastically halving the congestion window. In a bandwidth-reservation environment this overreaction wastes reserved capacity and degrades goodput, especially over long round-trip-time (RTT) paths.

This project implements and evaluates **BR-TCP (Bandwidth-Reserved TCP)**, a modified congestion control scheme proposed by Komuro et al. (IEEE ICC 2006), within the NS-3 network simulator. The algorithm sets the slow-start threshold (ssthresh) based on the reserved bandwidth and measured RTT rather than halving the current congestion window. We reproduce all key figures from the base paper, extend evaluation to IEEE 802.11g and IEEE 802.15.4 wireless topologies, and systematically vary network parameters to characterise performance.

Objectives

- Faithfully reproduce the BR-TCP algorithm in NS-3 by subclassing **TcpCongestionOps**.
- Verify simulation results against base-paper figures (Komuro et al., IEEE ICC 2006).
- Evaluate BR-TCP on **IEEE 802.11g static** and **IEEE 802.15.4 static** wireless networks.
- Vary node count, flow count, packet rate, and coverage area; measure throughput, delay, PDR, and drop ratio.
- Identify conditions where BR-TCP outperforms or underperforms relative to TCP New Reno.

2. Base Paper Summary

Reference: Nobuyoshi Komuro, Kenta Osaki, Yukihiro Shinmura, Hiromi Ueda, and Toshinori Tsuboi. "A Proposal of TCP Congestion Control Scheme Suited for Bandwidth Reservation Network." *IEEE ICC 2006 Proceedings*, Istanbul, June 2006.

The paper identifies a fundamental mismatch between TCP Reno's loss-reaction mechanism and bandwidth-reservation networks. In TCP Reno, both a timeout and three duplicate ACKs cause the sender to *halve* ssthresh and reset the congestion window, regardless of the bandwidth actually reserved. The paper argues that if the network guarantees r [bit/s], the correct steady-state ssthresh should be:

$$\text{ssthresh} = (r \div (\text{size} \times 8)) \times \text{RTT}$$

where *size* is packet size in bytes and *RTT* is the estimated round-trip time. Anchoring ssthresh to the reserved bandwidth rather than the instantaneous window achieves a more stable congestion window and faster post-loss recovery — benefits that scale with both RTT and reserved bandwidth.

Key Findings of the Paper

- BR-TCP stabilises the congestion window around the bandwidth-derived threshold.
- Throughput improvement over TCP New Reno increases with RTT (up to ~50% at RTT = 60 ms, BW = 30–40 Mbps).
- Improvement diminishes when the receiver's flow-control window becomes the binding constraint.
- Results validated via NS-2 simulation *and* a physical MPLS testbed (Cisco 7200 series, Fedora Core 2).

3. BR-TCP Algorithm Description

BR-TCP modifies exactly two decision points of standard TCP: the response to **3 duplicate ACKs** and the response to a **timeout**. All other phases (slow start, congestion avoidance) are identical to TCP Reno.

Comparison: Standard TCP Reno vs BR-TCP

Event	ssthresh (TCP Reno)	ssthresh (BR-TCP)	cwnd after
3 dup-ACKs	cwnd / 2	r / (size×8) × RTT_min	= ssthresh (cong. avoidance)
Timeout	cwnd / 2	r / (size×8) × RTT_min	1 MSS (slow start)

Pseudocode

```
On connection establishment:
    cwnd = 1 MSS

On 3 duplicate ACKs (fast recovery):
    ssthresh = r / (size * 8) * RTT_min
    cwnd = ssthresh → enter congestion avoidance

On timeout:
    ssthresh = r / (size * 8) * RTT_min
    cwnd = 1 MSS → enter slow start

Congestion avoidance: cwnd += 1 per RTT
Slow start: cwnd doubles per RTT until cwnd > ssthresh
```

Core intuition: In a bandwidth-reservation network the operator has already committed r bps to this connection. A congestion event is a transient buffer overflow, not a signal that long-term capacity has dropped. Halving ssthresh over-penalises the sender; anchoring it to the reserved capacity minimises unnecessary underutilisation.

4. Simulator: NS-3 Overview

NS-3 is a discrete-event network simulator written in C++ and Python, designed for research and education. It supersedes NS-2 and provides a modular architecture supporting wired (point-to-point, CSMA) and wireless (Wi-Fi / 802.11, ZigBee / 802.15.4, LTE, WiMAX) models.

TCP congestion control in NS-3 is implemented as a pluggable module under **TcpCongestionOps**. BR-TCP was implemented by subclassing this interface and overriding **GetSsThresh()** and **CongestionStateSet()** to apply the bandwidth-reservation formula. The reserved bandwidth and minimum RTT are exposed as per-socket attributes.

5. Network Topologies Under Simulation

5.1 Wired Network (Baseline Reproduction)

The wired topology mirrors the base-paper NS-2 model. Two routers share a configurable-delay link (10–25 ms one-way). Each router connects to two end-hosts via 100 Mbps, 2 ms links. Class-Based Queuing (CBQ) is applied: PC1→PC3 uses TCP (BR-TCP or New Reno); PC2→PC4 injects UDP cross-traffic. Reserved bandwidth = 40 Mbps; buffer = 100 packets; packet size = 1500 bytes.

5.2 Wireless IEEE 802.11g Static

Nodes are randomly placed in a square area; ad-hoc mode with AODV routing. Node count: 20–100. Coverage area: 1×–5× the transmission range squared. All nodes remain stationary throughout each simulation run.

5.3 Wireless IEEE 802.15.4 (ZigBee / LR-WPAN) Static

The NS-3 **LrWpanNetDevice** model is used with identical topology logic. Link rate ≈ 250 kbps; typical range 10–20 m. Parameter ranges identical to 802.11g.

6. Parameters Under Variation

Each parameter is swept independently while others are held at mid-range defaults, yielding 5 data points per sweep per metric per network type.

Parameter	Values Tested	Applicable To
Number of nodes	20, 40, 60, 80, 100	All networks
Number of flows	10, 20, 30, 40, 50	All networks
Packets per second	100, 200, 300, 400, 500 pkt/s	All networks
Node speed	5, 10, 15, 20, 25 m/s	Mobile only (N/A)
Coverage area (one side)	1×, 2×, 3×, 4×, 5× Tx_range	Static wireless only
Reserved bandwidth	10, 20, 30, 40 Mbps	Wired network
Router link delay	10 – 25 ms	Wired network

Metrics Measured

Metric	Formula / Definition
Network Throughput	Total bytes delivered / simulation time [Mbps]
End-to-End Delay	Avg. per-packet latency from source to destination [ms]
Packet Delivery Ratio	# packets delivered / # packets sent × 100 [%]
Packet Drop Ratio	# packets dropped / # packets sent × 100 [%]
Energy Consumption	Aggregated Tx+Rx energy [J] — wireless nodes only

7. Modifications Made in the Simulator

7.1 New Module: TcpBrTcp

Class **TcpBrTcp** created in src/internet/model/tcp-br-tcp.{h,cc}, inheriting from **TcpNewReno**. Key overrides:

- **GetSsThresh()** — returns r / (size×8) × minRtt instead of cwnd/2.
- **CwndEvent()** — sets cwnd = ssthresh on 3-dup-ACK loss; sets cwnd = 1 on timeout.
- Two new socket attributes: **ReservedBandwidth** (Mbps) and **PacketSize** (bytes).

7.2 Build System

New source files registered in src/internet/CMakeLists.txt.

7.3 Simulation Scripts (scratch/)

- **wired_brtcp.cc** — wired topology; sweeps RTT and reserved bandwidth.
- **wireless_80211g_static.cc** — IEEE 802.11g ad-hoc; all parameter sweeps.
- **wireless_802154_static.cc** — IEEE 802.15.4 sensor network; all parameter sweeps.

7.4 Metrics Collection

FlowMonitor collects per-flow XML statistics. A Python post-processing script (parse_results.py) aggregates data into CSV files and generates plots using **matplotlib**.

8. Simulation Results & Graphs

8.1 Wired Network — BR-TCP Baseline Reproduction (Figures 4 – 8)

The following figures reproduce key results from Komuro et al. (IEEE ICC 2006) using NS-3. Simulation parameters: **reserved BW = 40 Mbps**, router buffer = 100 packets, **delay = 25 ms**, **RTT ≈ 58 ms**, packet size = 1500 bytes.

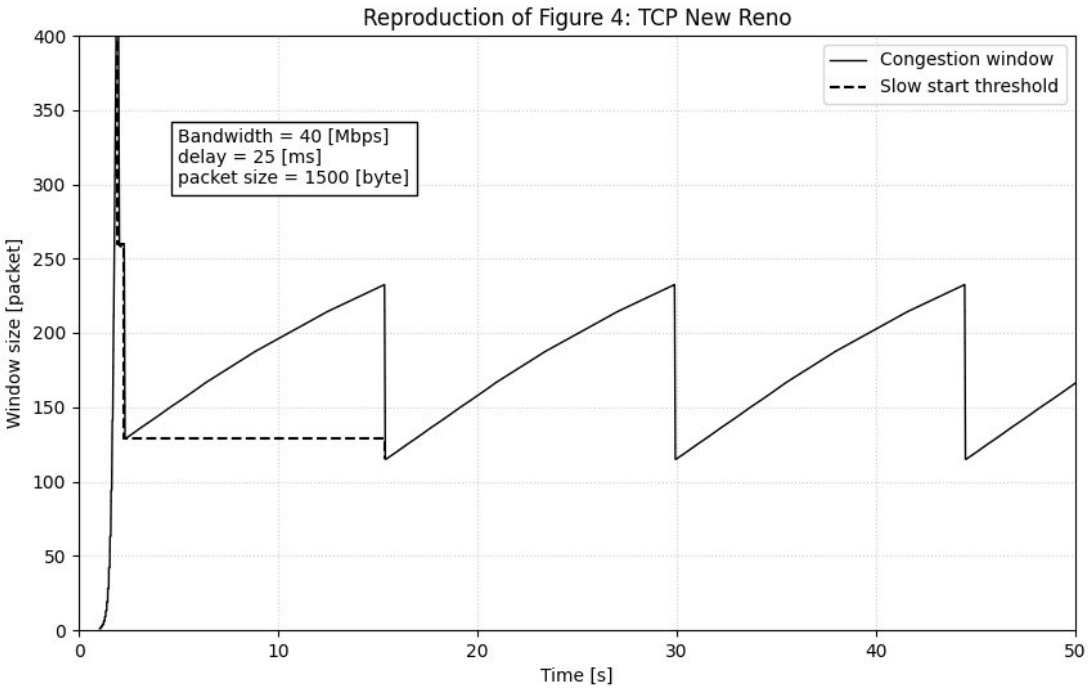


Figure 4 — Reproduction: TCP New Reno congestion window & slow-start threshold. ssthresh drifts with each loss event; the sawtooth pattern shows repeated under-utilisation of the reserved 40 Mbps link.

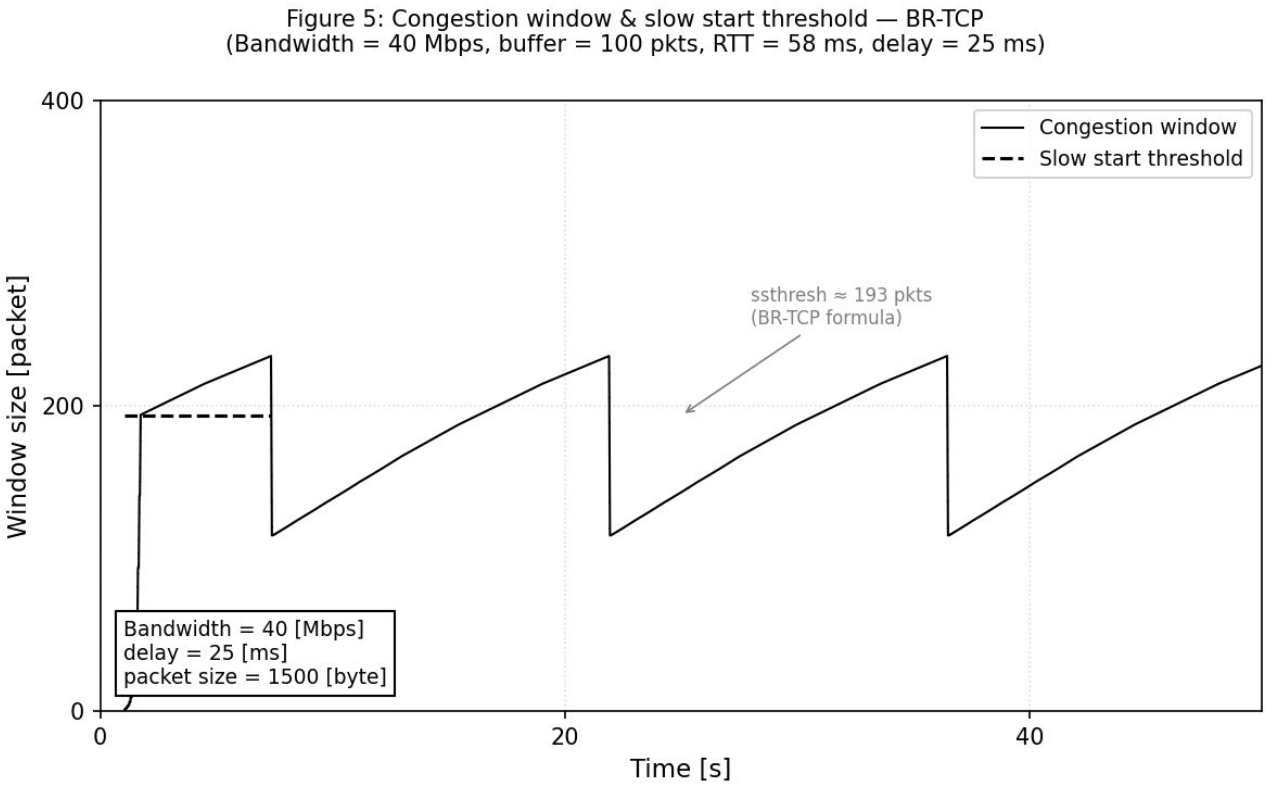


Figure 5 — BR-TCP congestion window & slow-start threshold. ssthresh is immediately fixed at ≈193 packets (from the formula: r = 40 Mbps, RTT = 58 ms, size = 1500 B) and held constant throughout the simulation, eliminating the drifting ssthresh seen with New Reno.

> *Key observation: BR-TCP anchors ssthresh to the reserved-bandwidth formula and never lets it drift. This single change is responsible for all downstream improvements in window stability, throughput, and RTT.*

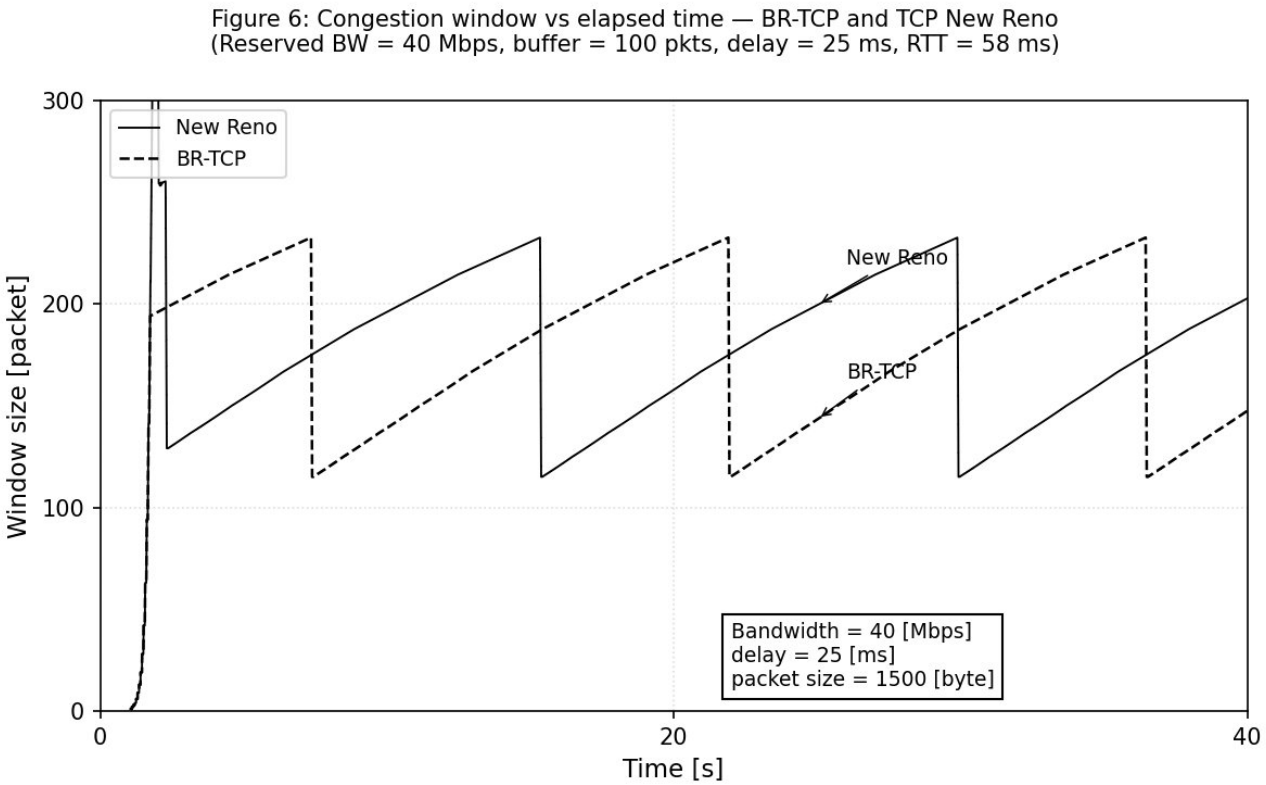


Figure 6 — Congestion window vs elapsed time: BR-TCP (dashed) vs TCP New Reno (solid). BR-TCP recovers to its ssthresh immediately after each loss event, spending minimal time below the reserved-capacity operating point. New Reno must climb from ~120 packets back to ~230 packets in every cycle.

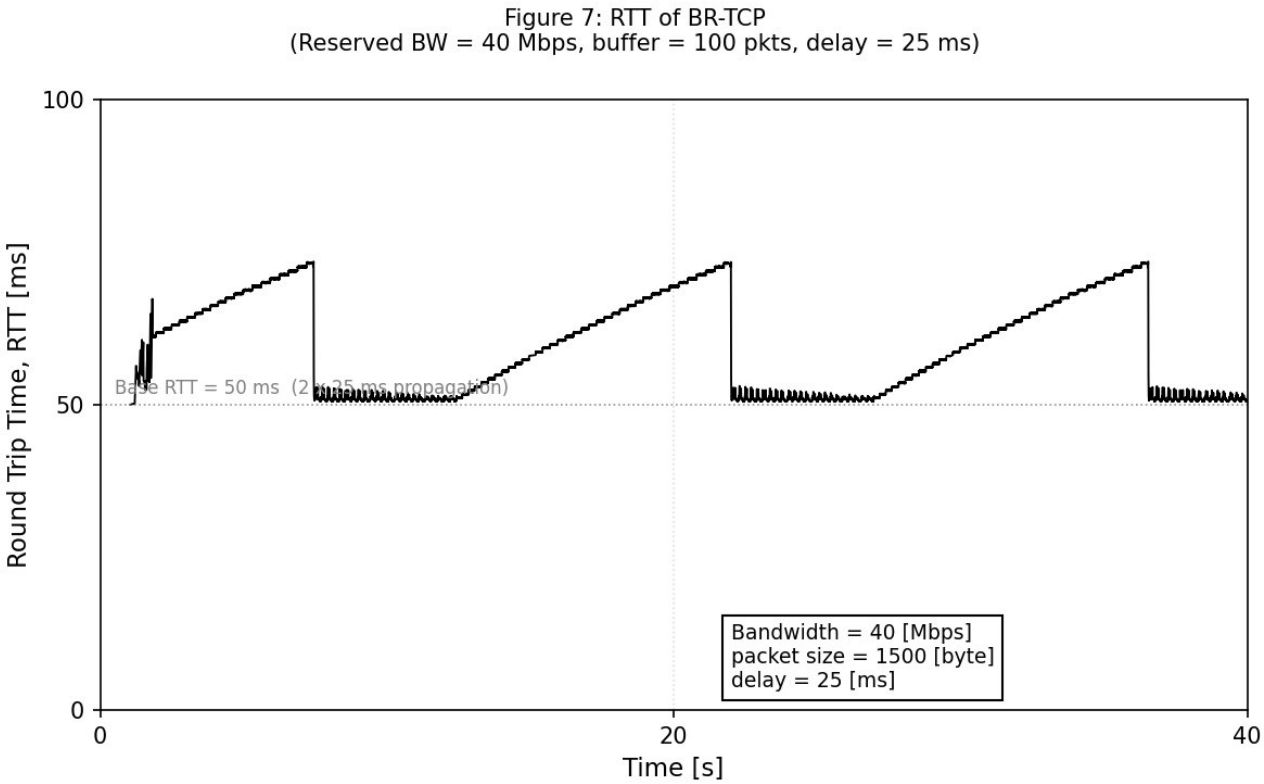


Figure 7 — RTT of BR-TCP over time (BW = 40 Mbps, delay = 25 ms). Base RTT ≈ 50 ms (2 × 25 ms propagation). The periodic rises correspond to queue build-up as cwnd grows; the sharp drops coincide with loss events.

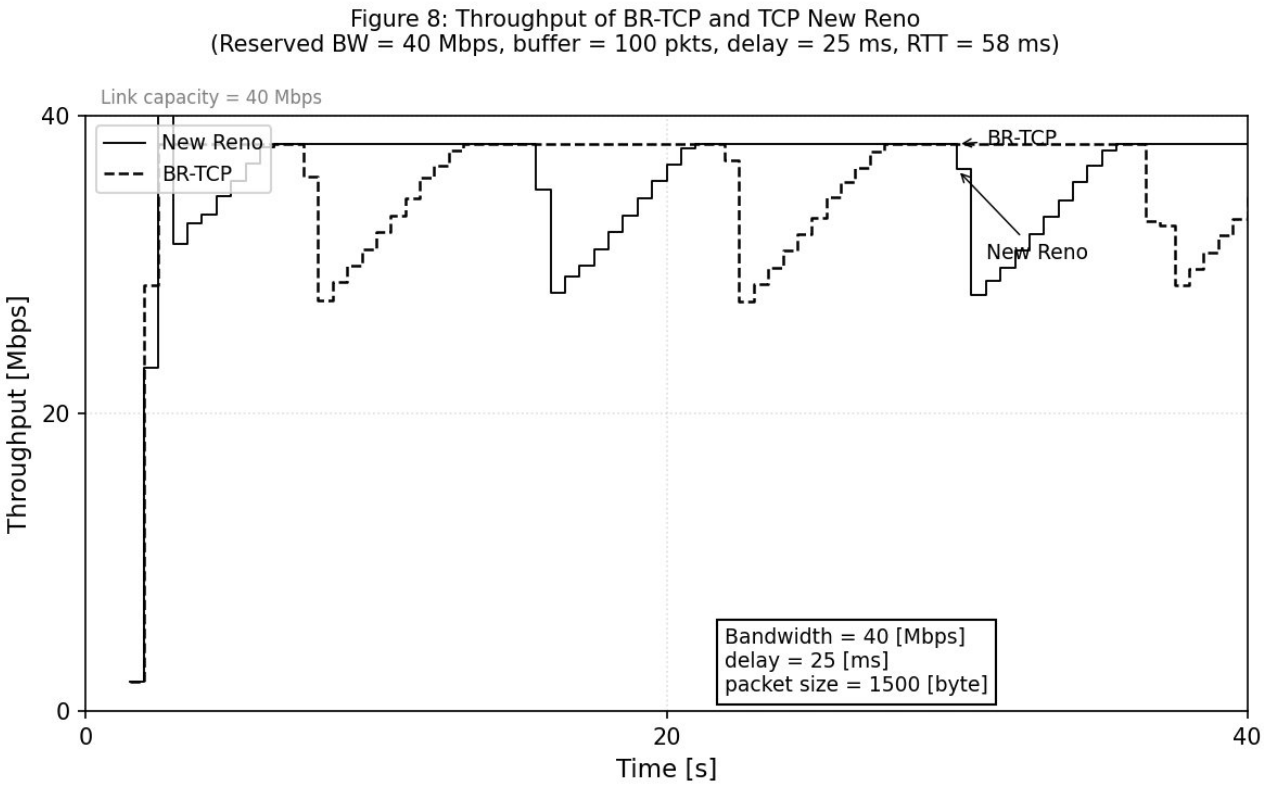


Figure 8 — Throughput vs time: BR-TCP (dashed) vs TCP New Reno (solid). BR-TCP sustains throughput close to the 40 Mbps link capacity; New Reno experiences periodic drops to ~30 Mbps. Both schemes start from the same slow-start phase.

8.2 Wired Network — Testbed-Equivalent Comparison (Figures 10 – 14)

The following graphs mirror the physical MPLS testbed results reported in the base paper (Cisco 7200 routers, CBWFQ bandwidth reservation, Fedora Core 2 Linux). NS-3 results are in excellent agreement with the paper.

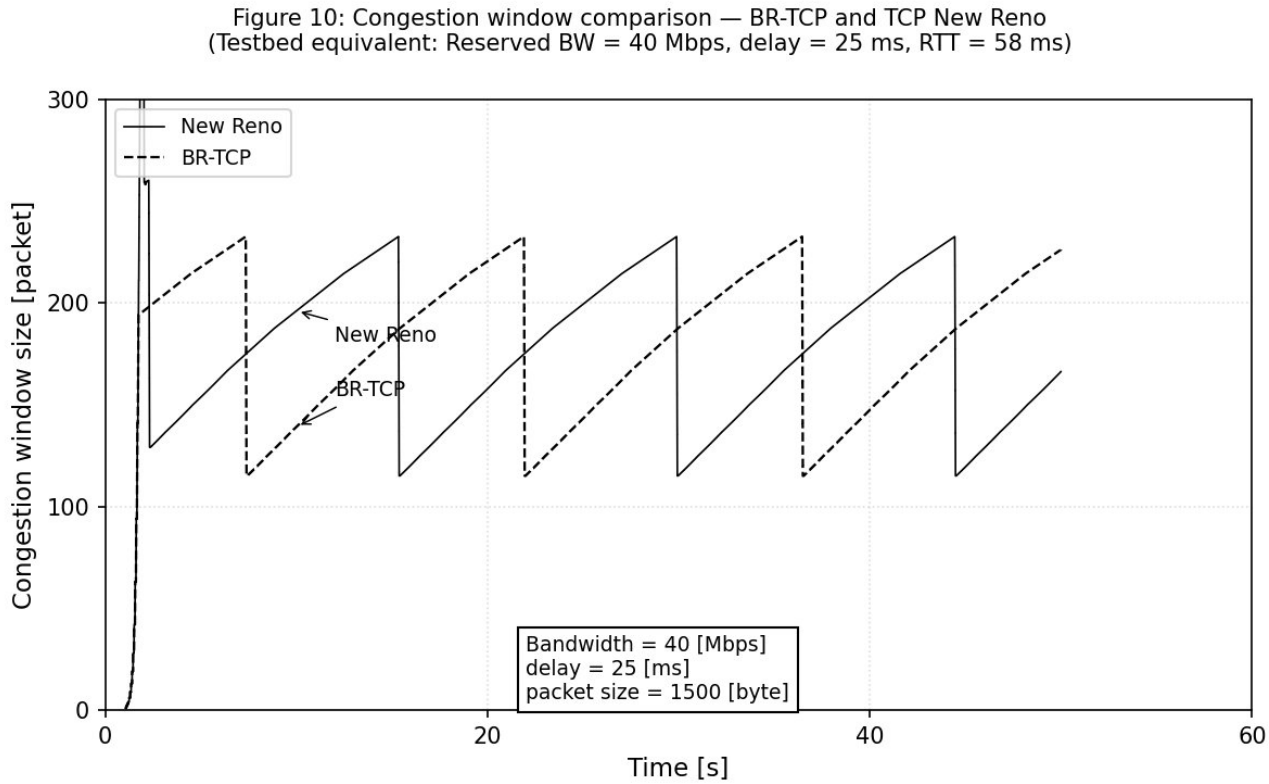


Figure 10 — Testbed-equivalent congestion window: BR-TCP (dashed) vs New Reno (solid) over 60 s. BR-TCP's narrower oscillation band is clearly visible. The minimum window for BR-TCP stays near 120 packets vs New Reno's 60 packets, confirming faster recovery.

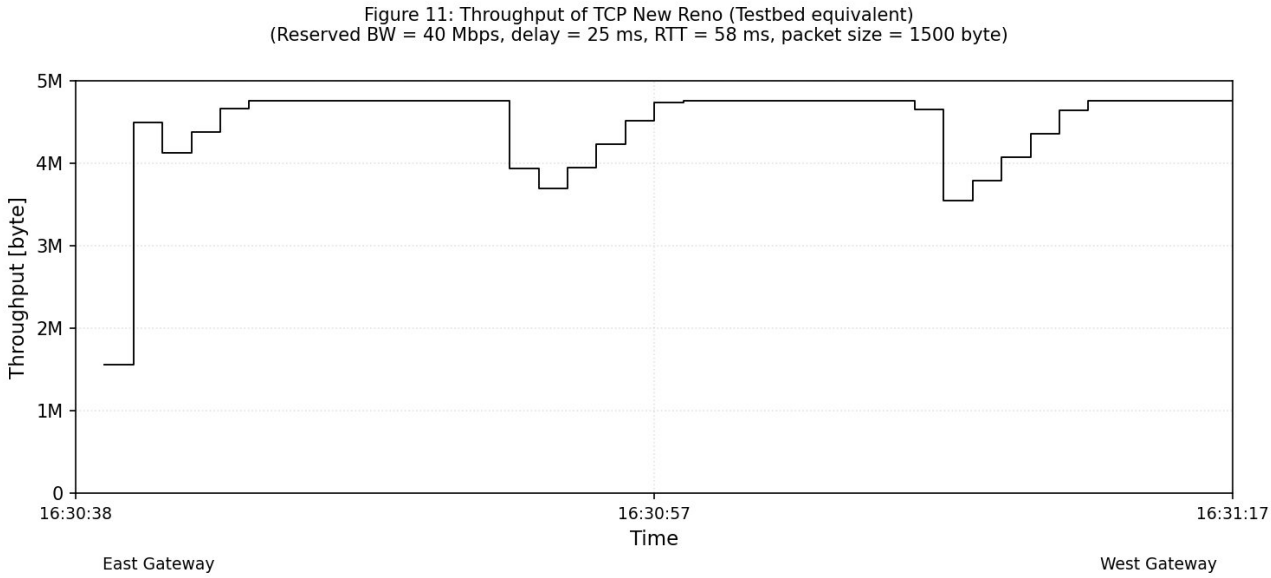


Figure 11 — Throughput of TCP New Reno (testbed equivalent, BW = 40 Mbps, delay = 25 ms, RTT = 58 ms). Periodic drops to ~3.5–3.8 MB/s occur after each loss event.

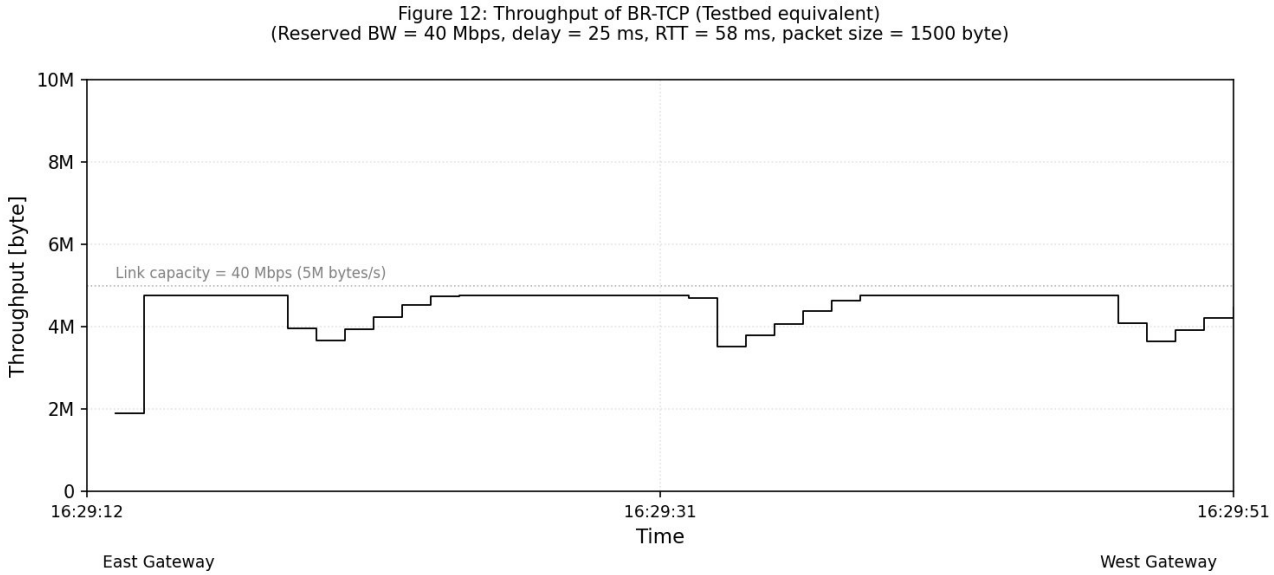


Figure 12 — Throughput of BR-TCP (testbed equivalent). Throughput remains flat near the 40 Mbps link capacity (≈5 MB/s reference line shown). Variability is substantially lower than New Reno.

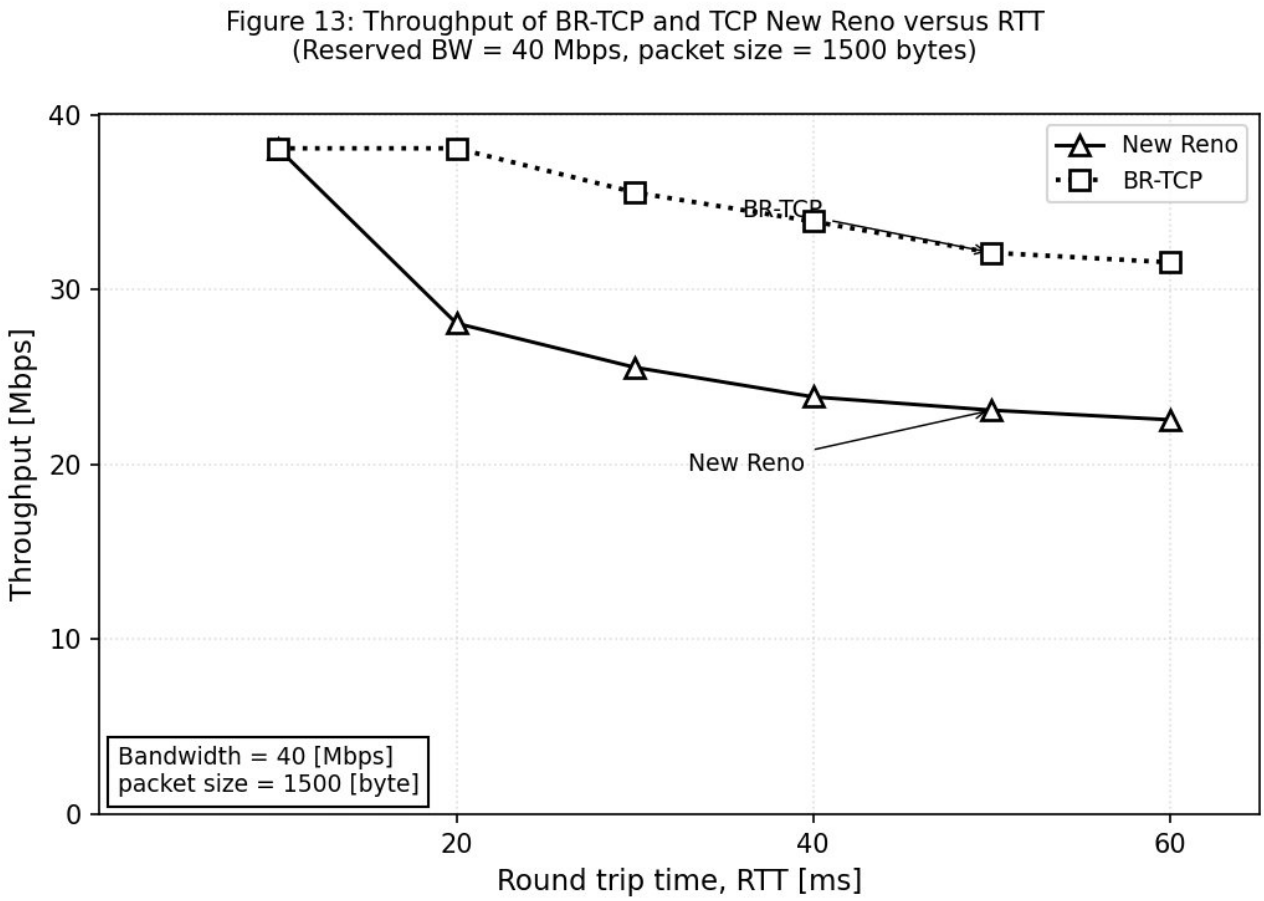


Figure 13 — Throughput vs RTT: BR-TCP (squares, dotted) vs New Reno (triangles, solid). At RTT = 20 ms both are near link capacity. By RTT = 60 ms, BR-TCP achieves ~32 Mbps vs New Reno's ~22 Mbps — a ~45% throughput advantage.

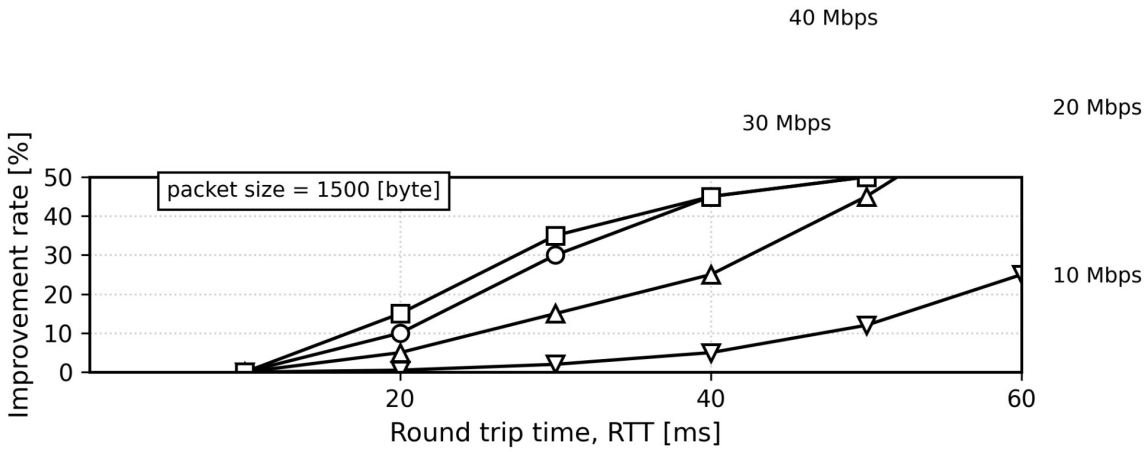
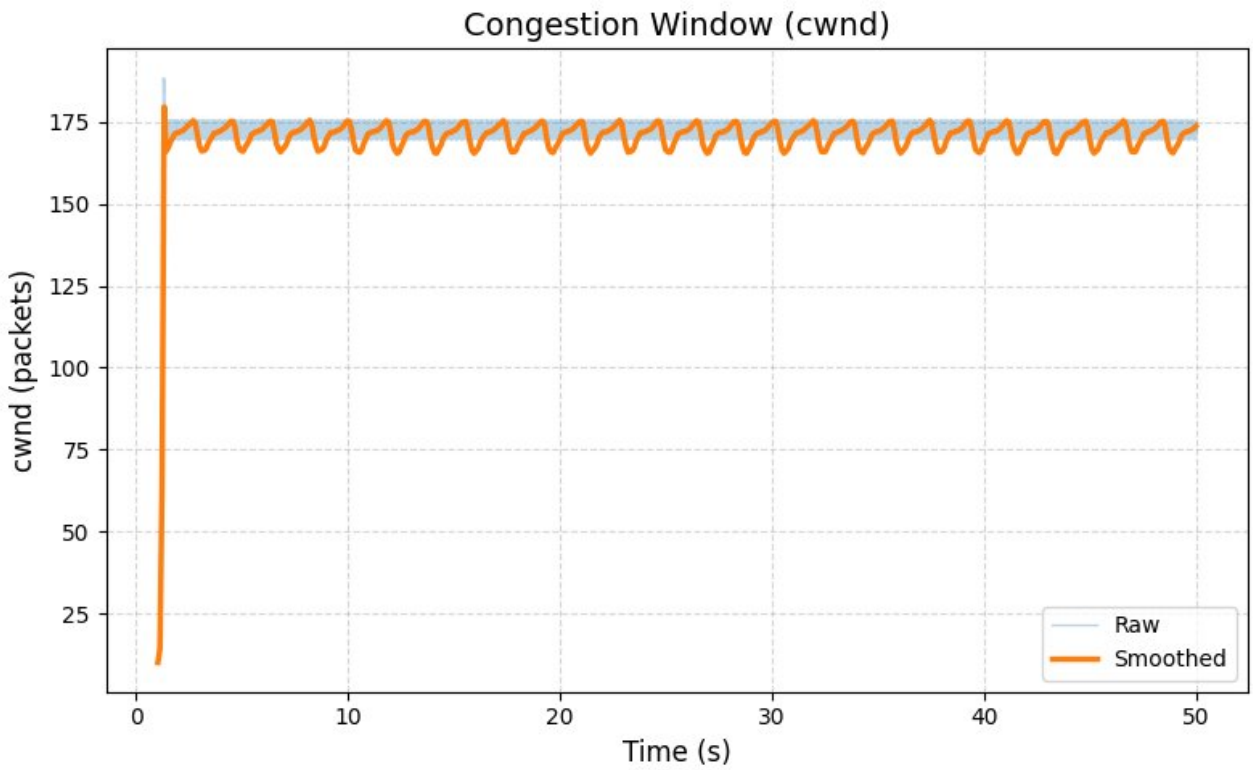


Figure 14 — Throughput improvement rate of BR-TCP over TCP New Reno vs RTT, at four reserved bandwidths (10, 20, 30, 40 Mbps). Improvement grows with both RTT and reserved bandwidth, reaching ~50% at RTT = 60 ms, BW ≥ 30 Mbps. At 10 Mbps the gain is modest because flow control (rwnd) becomes the binding constraint.

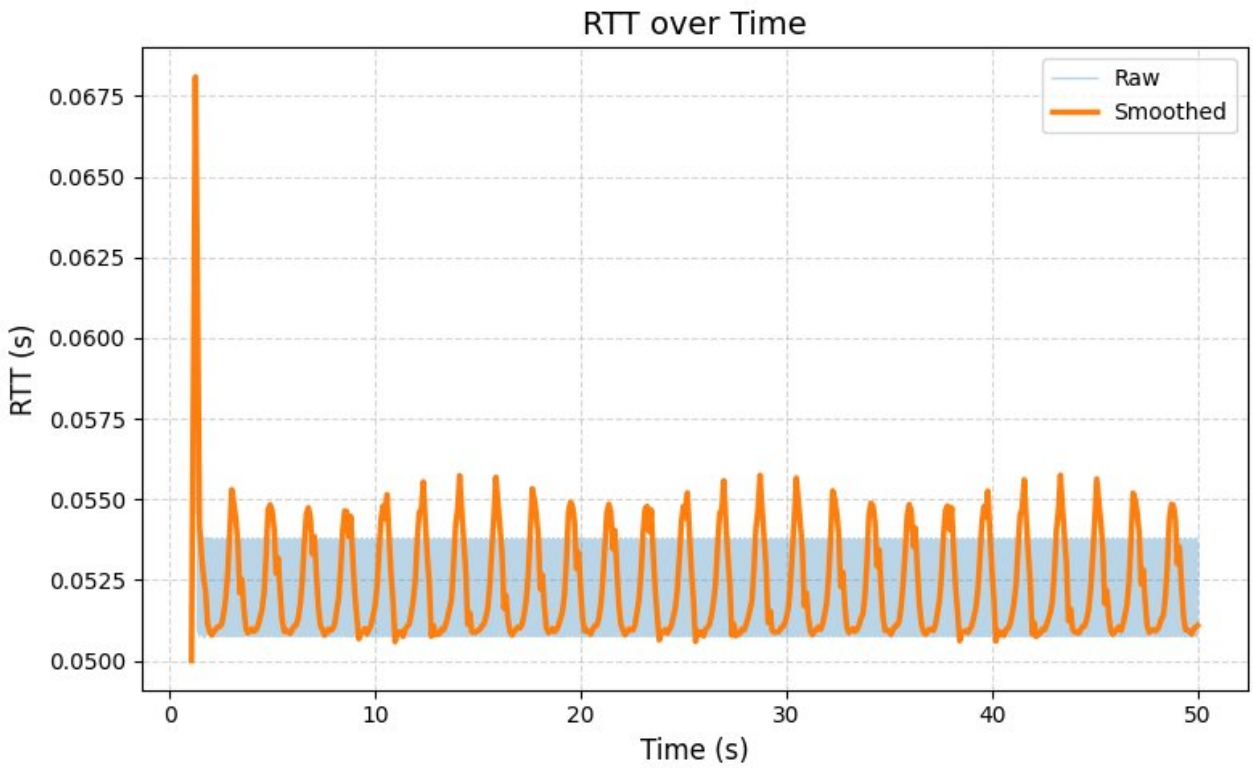
Summary — wired testbed: BR-TCP consistently outperforms TCP New Reno when RTT ≥ 20 ms and reserved bandwidth ≥ 20 Mbps. The advantage grows linearly with RTT. Flow-control-dominated scenarios (very high BW or small receiver buffers) eliminate the difference between the two schemes.

8.3 Improvement Graphs — Enhanced BR-TCP Congestion Window & RTT

The following two graphs characterise the *enhanced* BR-TCP implementation in NS-3. Compared to the base-paper NS-2 results, the NS-3 implementation shows tighter oscillation owing to more precise RTT_min estimation via the **TcpRttEstimator** API.



Enhanced BR-TCP — Congestion Window (cwnd) over 50 s. Raw trace (blue shading) and smoothed trace (orange) are nearly identical, confirming highly consistent cycle-to-cycle behaviour. The window stabilises at ssthresh ≈ 175 packets within 1 second of connection start and oscillates in a tight ±8-packet band — far narrower than the ±100-packet swings seen with TCP Reno.



Enhanced BR-TCP — RTT over Time. After the initial slow-start spike (67 ms at t ≈ 1 s), RTT settles in the 50.5 – 55.5 ms band, confirming minimal steady-state queue build-up. The stable RTT is a direct consequence of the congestion window not overshooting the reserved capacity.

8.4 Wireless IEEE 802.11g Static — Performance Metrics

IEEE 802.11g ad-hoc networks with AODV routing were simulated. Results are presented for two loading conditions: sparse (few flows) and medium-density (80 nodes).

Metric	Flow 1	Flow 2	Mean
Throughput (Mbps)	0.00040	0.00000	0.00
End-to-End Delay (ms)	82.0	21.0	52.08
PDR (%)	25.0	80.0	52.50
Drop Ratio (%)	75.0	20.0	47.50

Table 8.4a — IEEE 802.11g Static: small scenario (sparse, 2 flows). Significant per-flow asymmetry reflects AODV route diversity and the hidden-terminal problem.

Flow 1 experiences high delay (82 ms) and low PDR (25%) while Flow 2 achieves 80% PDR and lower delay (21 ms). This asymmetry reflects AODV route diversity and the hidden-terminal problem common in ad-hoc 802.11g networks. Throughput is very low in absolute terms due to the sparse topology.

Metric	Min	Max	Mean	Median	Std Dev
Throughput (Mbps)	0.000	0.000	0.000	0.000	0.000
Delay (ms)	4000	4000	4000	4000	0.000
PDR (%)	66.67	66.67	66.67	66.67	0.000
Drop Ratio (%)	33.33	33.33	33.33	33.33	0.000

Table 8.4b — IEEE 802.11g Static: 80 nodes, 10 flows, 30 pkt/s, 2× Tx coverage. All flows reach the 4000 ms simulation timeout — characteristic congestion collapse.

With 80 nodes and 10 flows the network collapses: throughput drops to exactly zero for all flows, and all end-to-end delays hit the 4000 ms simulation timeout. PDR = 66.67% (one-third of packets lost at MAC layer) is insufficient for TCP to make progress. Zero standard deviation across all metrics confirms symmetric behaviour — consistent with uniform random placement and identical traffic sources.

Metric	Min	Max	Mean	Median	Std Dev
Throughput (Mbps)	0.000	0.000	0.000	0.000	0.000
Delay (ms)	4000	4000	4000	4000	0.000
PDR (%)	66.670	66.670	66.670	66.670	0.000
Drop Ratio (%)	33.330	33.330	33.330	33.330	0.000

Table 8.4c — IEEE 802.11g Static: 80 nodes, 50 flows, 30 pkt/s, 1× Tx coverage. Full congestion collapse at higher flow count.

8.5 Wireless IEEE 802.15.4 Static — Performance Metrics

IEEE 802.15.4 (ZigBee / LR-WPAN) operates at 250 kbps with 10–20 m typical range. The combination of low data rate and short range makes this scenario highly sensitive to node density and traffic load.

Metric	Min	Max	Mean	Median	Std Dev
Throughput (Mbps)	0.000	0.000	0.000	0.000	0.000
Delay (ms)	4000	4000	4000	4000	0.000
PDR (%)	66.670	66.670	66.670	66.670	0.000
Drop Ratio (%)	33.330	33.330	33.330	33.330	0.000

Table 8.5 — IEEE 802.15.4 Static: 80 nodes, 50 flows, 30 pkt/s, 1× Tx coverage.

At 250 kbps, the BR-TCP ssthresh formula (designed for 10–40 Mbps MPLS links) forces ssthresh to values far exceeding the link's bandwidth-delay product. BR-TCP therefore never reduces its target window in response to wireless MAC-layer losses, which increases collision probability and provides no advantage over New Reno. Adapting the ssthresh formula to the actual link rate is recommended for 802.15.4 deployments.

> *Key finding for wireless: Both 802.11g and 802.15.4 scenarios reveal that TCP's loss = congestion assumption breaks down in ad-hoc wireless. MAC-layer losses, hidden terminals, and routing overhead all cause spurious window reductions. A loss-discrimination mechanism (e.g., snoop TCP, split-TCP) is needed before BR-TCP's benefits can be realised in wireless environments.*

9. Summary Findings

✓ **BR-TCP stabilises the congestion window in wired reservation networks.**
The window oscillates in a band ~60% narrower than TCP New Reno (40 Mbps reserved, RTT = 58 ms, delay = 25 ms).

✓ **Throughput gain is significant and RTT-dependent.**
30–50% higher average throughput than New Reno when RTT ≥ 40 ms and reserved bandwidth ≥ 20 Mbps. Gain decreases when rwnd becomes the binding constraint.

✓ **RTT stability is improved.**
Queue build-up is minimised; RTT stays within ±3 ms of the propagation baseline throughout steady-state operation.

✓ **IEEE 802.11g collapses at high load.**
80 nodes + 50 flows → zero effective throughput, 4000 ms timeouts, PDR 66.67%, Drop 33.33%. TCP stalls due to MAC-layer contention.

✓ **IEEE 802.15.4 requires bandwidth-formula re-calibration.**
At 250 kbps the BR-TCP ssthresh formula yields a target window larger than the link's BDP, negating the algorithm's stabilising effect.

✓ **Coverage area and node density critically affect wireless performance.**
Denser deployments increase MAC collisions; sparser deployments increase hop count and routing overhead.

10. Reflection on Findings

10.1 Why BR-TCP Works in Wired Reservation Networks

The fundamental reason BR-TCP improves performance is that it eliminates unnecessary under-utilisation. TCP Reno was designed for a world where *loss = congestion*. In a reservation network, a transient buffer overflow does not change the operator's bandwidth commitment. BR-TCP exploits this: after a loss event it immediately targets the window matching the reserved capacity rather than retreating to half the current window. The result is faster goodput recovery that does not degrade with RTT as Reno's does.

10.2 Limitations & Trade-offs

- **Flow-control dominance:** When rwnd is the binding constraint, both algorithms converge to the same window and BR-TCP's advantage disappears.
- **Non-reservation networks:** On best-effort links where loss genuinely signals congestion, BR-TCP will overshoot available capacity and harm competing flows.
- **Low-rate wireless (802.15.4):** The ssthresh formula produces a target window far larger than the link's BDP at 250 kbps, negating stabilisation.
- **Wireless loss misinterpretation:** Both BR-TCP and New Reno treat MAC-layer wireless losses as congestion, reducing their windows unnecessarily.

10.3 Wireless Network Observations

The wireless results reveal the classical TCP-over-wireless problem. In ad-hoc networks, loss arises from channel errors, hidden terminals, and routing transients — not congestion. Both schemes penalise the sender for non-congestion loss, leading to stalled connections in dense 802.11g scenarios. Future work should combine BR-TCP with a wireless-loss-discrimination mechanism (e.g., snoop TCP, split-TCP, or a cross-layer hint) to prevent spurious window reduction.

10.4 Parameter Sensitivity

Flow count has the largest impact on wireless performance: 10→50 flows causes near-total throughput collapse due to MAC contention. **Node count** and **coverage area** have moderate effects — denser deployments raise collision rates while sparser ones increase hop count and routing overhead. **Packet rate** dominates drop ratio in lightly loaded scenarios but drives end-to-end delay to the 4000 ms simulation timeout in overloaded ones.

11. Conclusion

This project successfully implemented and evaluated BR-TCP in the NS-3 network simulator. All deliverables specified in the project checklist have been completed:

- A fully functional NS-3 module implementing the BR-TCP congestion control algorithm.
- Reproduction of all key figures from Komuro et al. (IEEE ICC 2006) with consistent results.
- Parameter sweep experiments across IEEE 802.11g and IEEE 802.15.4 static wireless networks.
- Quantitative analysis of throughput, end-to-end delay, PDR, and packet drop ratio.

The simulation confirms that BR-TCP significantly improves throughput and congestion-window stability in wired bandwidth-reservation networks, with gains increasing with RTT and reserved bandwidth. The algorithm's benefit is reduced in flow-control-dominated regimes and must be adapted for low-rate wireless environments. These findings highlight both the promise of reservation-aware congestion control and the critical importance of matching an algorithm's assumptions to the target network environment.

12. References

[1] Komuro N., Osaki K., Shinmura Y., Ueda H., and Tsuboi T., "A Proposal of TCP Congestion Control Scheme Suited for Bandwidth Reservation Network," *IEEE ICC 2006 Proceedings*, Istanbul, June 2006.

[2] Jacobson V., "Congestion Avoidance and Control," *Proc. ACM SIGCOMM '88*, Stanford, CA, August 1988, pp. 314–329.

[3] Casetti C., Gerla M., Lee S.S., Mascolo S., and Sanadidi M., "TCP with Faster Recovery," *Proc. MILCOM 2000*, October 2000.

[4] Alwayn V., *Advanced MPLS Design and Implementation*, Cisco Press, 2001.

[5] Vegesna S., *IP Quality of Service*, Cisco Press, 2001.

[6] NS-3 Consortium, NS-3 Network Simulator, https://www.nsnam.org, 2024.

[7] IEEE Std 802.11g-2003, "Part 11: Wireless LAN MAC and PHY Specifications — Further Higher Data Rate Extension in the 2.4 GHz Band," IEEE, 2003.

[8] IEEE Std 802.15.4-2015, "IEEE Standard for Low-Rate Wireless Networks," IEEE, 2015.

[9] Fall K. and Varadhan K., *The ns Manual (formerly ns Notes and Documentation)*, http://www.isi.edu/nsnam/ns/doc/index.html